

EXPLICACIÓ LAB4 - DAVID PÉREZ CRUZ

CODI TT.C:

En aquesta part del codi es defineixen constants com TEAMNAME, ENIGMA3, SILVERKEY i GOLDKEY, que s'utilitzaran més endavant per a la connexió amb el servidor. A continuació, es comprova que el programa rebi exactament dos arguments (M i N) i que siguin enters majors que 0 mitjançant la funció nombre_valid. Si no es compleixen aquestes condicions, es mostra un missatge d'error i el programa finalitza. Després, el codi intenta establir connexions amb el servidor per al control i per a les dades; si alguna fallida, es mostra un error i es surt del programa.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <fcntl.h>
#include "dummy4.h"

#define SERVNAME "SIOP-server"
#define SERVADDR "147.83.159.200"
#define SERVPORT 9999
#define CTRLPORT 9900
#define TEAMNAME "davidperezcruz" // Inserta entre comillas tu
nombre de equipo
#define ENIGMA3 "belters" // Inserta entre comillas la solución
del enigma 3
#define SILVERKEY "ab452" // Inserta SILVERKEY entre comillas
#define GOLDKEY "" // use only for SIOP Challenge: insert GOLDKEY
between ""

int nombre_valid(char *str) {
    int num = atoi(str);
    return num > 0;
}

int main(int argc, char *argv[]) {
    if (argc != 3) {
        fprintf(stderr, "Uso: ./tt M > 0 N > 0\n");
        exit(-1);
    }
}
```

```

    }

    if (!nombre_valid(argv[1]) || !nombre_valid(argv[2])) {
        fprintf(stderr, "Error: M y N deben ser enteros mayores
que 0.\n");
        exit(-1);
    }

    int ctrlfd, fd;
    int canalpipe[2];

    // Conectar al servidor
    ctrlfd = ctrlconnect(SERVADDR, CTRLPORT, ENIGMA3, TEAMNAME,
SILVERKEY, GOLDKEY);
    if (ctrlfd < 0) {
        perror("Error conectando al Capitol (canal de control)");
        exit(-1);
    }

    fd = connecta(SERVADDR, SERVPORT);
    if (fd < 0) {
        perror("Error conectando al Capitol (canal de datos)");
        exit(-1);
    }

    pipe(canalpipe); // 0 -> lectura, 1 -> escritura

```

Tot seguit, en aquesta part del codi, es crea un número de processos fills (replicants) igual a M, passant com a argument el valor de argv[2]. Per cada replicant, es redirigeix l'entrada estàndard per utilitzar el canal de dades (fd) i es tanca la connexió del pipe. A continuació, s'executa el programa replicant dins del procés fill. Si es produeix un error en crear un replicant, el programa finalitza.

Després, el programa espera que tots els replicants acabin la seva tasca i acumula els resultats obtinguts mitjançant la funció WEXITSTATUS. Si algun replicant falla, es mostra un error i el programa acaba.

Finalment, s'envia el resultat acumulat al Capitol mitjançant el canal de control, es llegeix la resposta del Capitol i es mostra per pantalla. Els descriptors de fitxer s'eliminen abans que el programa acabi.

```
// Crear replicantes
int M = atoi(argv[1]);
for (int i = 0; i < M; ++i) {
    int pid = fork(); // Crear un proceso hijo
    if (pid == 0) {
        // En el proceso hijo (replicant)
        close(0); // Redirige la entrada estándar
        dup(fd);
        close(canalpipe[0]);
        close(canalpipe[1]);
        close(fd);

        execlp("./replicant", "./replicant", argv[2], (char*)
0); // Ejecutar replicant
        perror("Error en el replicant");
        exit(-1);
    } else if (pid < 0) { // Error en la creación de procesos
        perror("Error creando un replicant");
        exit(-1);
    }
}

// Esperar a que todos los replicantes terminen
int acumula = 0; // Variable que acumula el número de
replicantes procesados
for (int i = 0; i < M; ++i) {
```

```

        int stat;
        wait(&stat);

        if (WIFEXITED(stat)) { // Si el proceso hijo terminó
correctamente
            acumula += WEXITSTATUS(stat);
        } else {
            printf("Replicant %d error.\n", i);
            exit(-1);
        }
    }

    /***** sending acum to Capitol *****/
    char buf[128];
    int nbytes = sprintf(buf, "%d", acumula); // Convertir acumula
a string
    write(ctrlfd, buf, nbytes); // Enviar el valor de acumula al
Capitol

    /***** reading answer from Capitol *****/
    char buffy[128];
    int len = read(ctrlfd, buffy, 128); // Leer la respuesta del
Capitol
    buffy[len] = '\0'; // Asegurarse de que el string esté bien
terminado
    printf("MISSION: %s\n", buffy); // Mostrar la respuesta

    // Cerrar los descriptores de archivo
    close(ctrlfd);
    close(fd);

    exit(0);

```

CODI REPLICANT.C:

En el codi del replicant, primer incloem la capçalera "dummy.h". A continuació, creem un buffer de 200 caràcters i comprovem que N sigui menor a 200. Si no ho és, mostrem un error i sortim amb exit(-1).

Si la validació és correcta, cridem dummy_init i creem la variable bytesllegits per llegir els bytes. Després, fem un bucle on llegim valors i els assignem a bytesllegits mentre aquest valor sigui major a 0.

Finalment, cridem dummy_comp per processar el temps. Si tot va bé, finalitzem amb dummy_exit i sortim amb exit(0) per indicar que el replicant ha acabat correctament.

```
#include "dummy4.h"
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
    int N = atoi(argv[1]);
    if (N > 200 || N < 0) {
        exit(-1);
    }

    char buffer[200];
    dummy_init(buffer, N);

    int bytesllegits;
    while ((bytesllegits = read(0, buffer, N)) > 0) {
        dummy_calc(buffer, bytesllegits);
    }

    dummy_end();
    exit(0);
}
```

CODI CAPITOL.C:

En el codi del capitol, primer incloem la capçalera "dummy3.h". A continuació, enviem una senyal SIGUSR1 al procés pare per indicar que estem llestos per començar.

Després, utilitzem un bucle per llegir caràcters de l'entrada estàndard. Per cada caràcter llegit, el mirem i el tornem a escriure a l'eixida estàndard.

Un cop acabat de llegir i escriure, tanquem l'entrada i sortida. Finalitzem el programa amb `exit(0)` per indicar que tot ha anat bé.

```
#include "dummy4.h"
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <stdlib.h>

int main() {
    kill(getppid(), SIGUSR1);

    char c;
    while (read(0, &c, 1) > 0) {
        write(1, &c, 1);
    }

    return 0;
}
```

COMPROVACIÓ DEL CODI:

```
print( "MISSION %s\n" % dummy )
david@david-Modern-14-C13M:~/2no/SIOP/lab4$ gcc -o tt tt.c -L. -ldummy4
david@david-Modern-14-C13M:~/2no/SIOP/lab4$ gcc -o capitol capitol.c -L. -ldummy4
david@david-Modern-14-C13M:~/2no/SIOP/lab4$ gcc -o replicant replicant.c -L. -ldummy4
david@david-Modern-14-C13M:~/2no/SIOP/lab4$ ./tt 2 3

*** CLIENT: Connecting server at [147.83.159.200]:[9900]

*** CHECKING: ENIGMA3 and SILVERKEY ...
[OK]: correct ENIGMA3 and SILVERKEY!!!

*** CLIENT: identification

*** CLIENT: sending teamname ...

[CAPITOL]: CLIENT davidperezcruz ALREADY REGISTERED

Waiting to read TimeBank....

*** CLIENT: Connecting server at [147.83.159.200]:[9999] ready to receive data
*** DUMMY: Init CLIENT dummy code...
*** DUMMY: Init CLIENT dummy code...
*** DUMMY: Exit CLIENT dummy code...
*** DUMMY: Exit CLIENT dummy code...
MISSION: SUCCESSFUL!!.GOLDKEY:0xdc192
```